Lesson 01 Demo 01

Creating CloudFormation Stack for Launching VM

Objective: To create CloudFormation stack for launching VM

Tools required: AWS CloudFormation

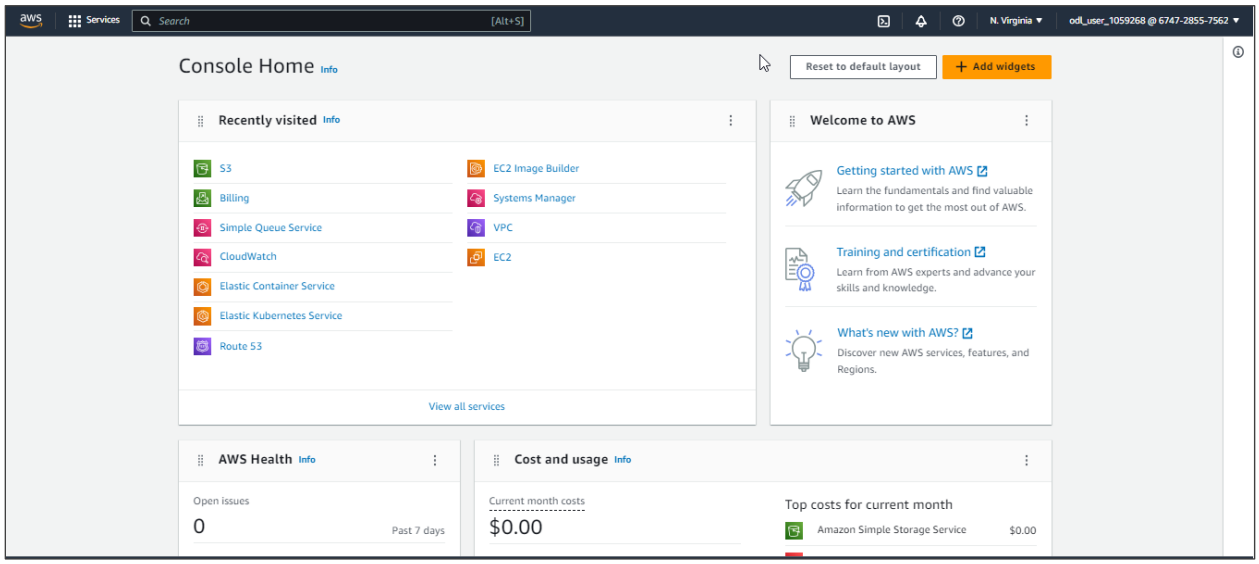
Prerequisites: None

Steps to be followed:

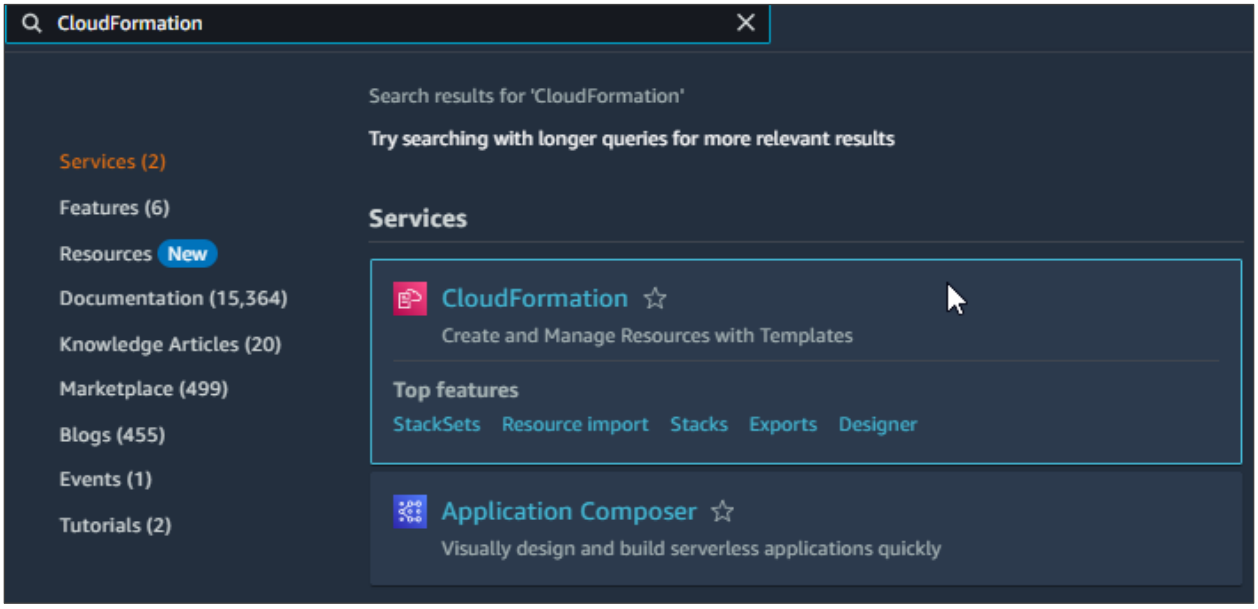
1. Log in to the AWS Console
2. Create a new CloudFormation template to launch a new VM

Step 1: Log in to the AWS Console

1.1 Click on the Practice Labs tab and log in to the **AWS Console** from the **AWS Labs**

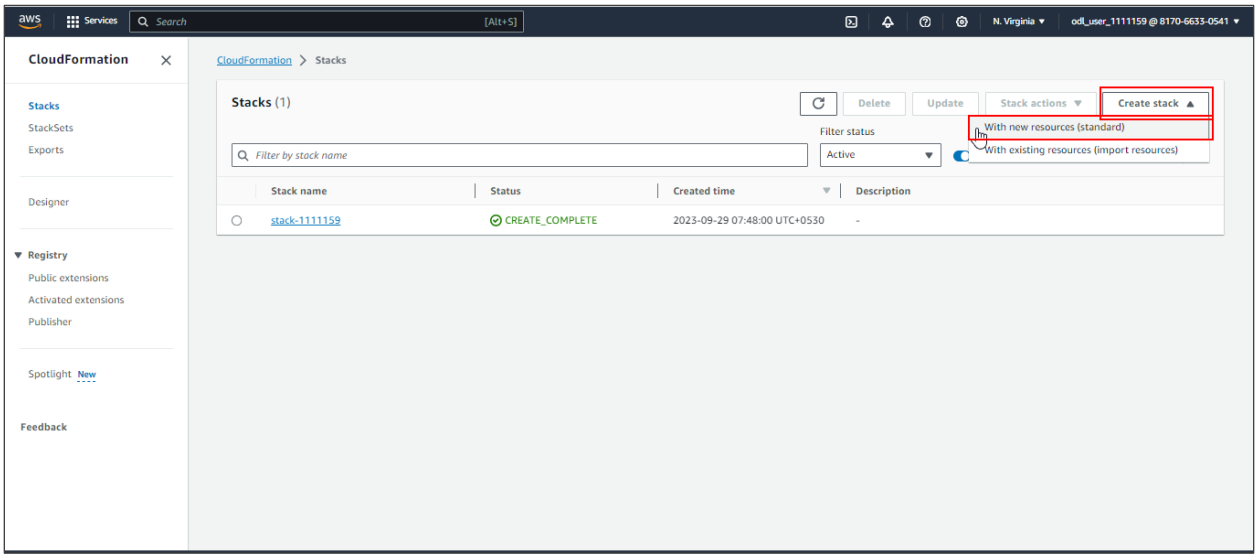


1.2 Search for and select CloudFormation in the AWS Console

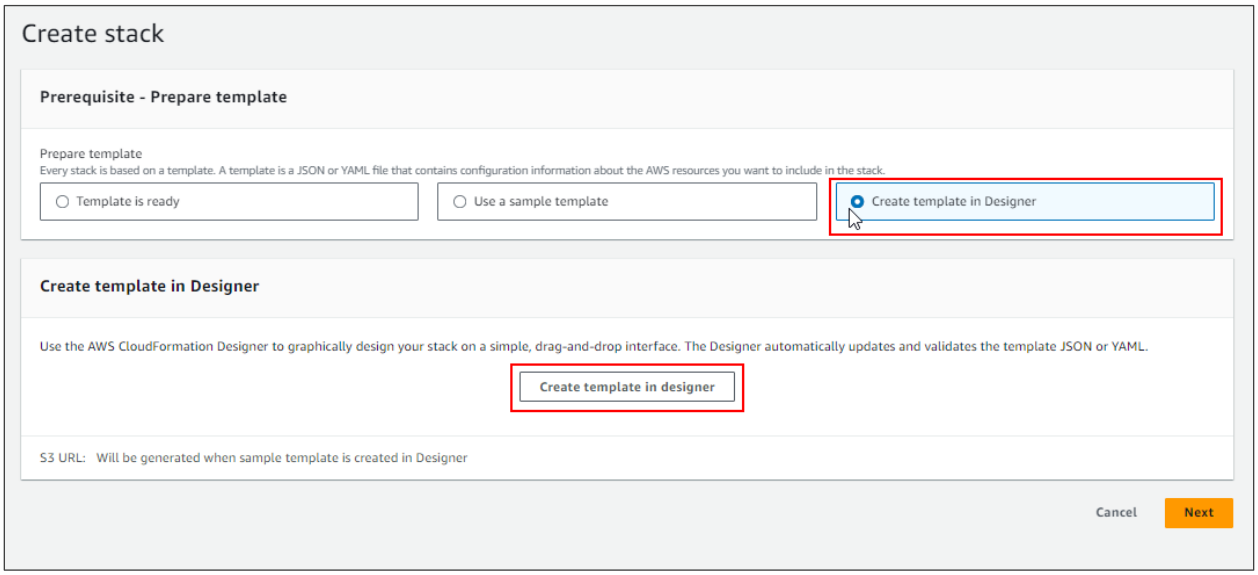


Step 2: Create a new CloudFormation template to launch a new VM

2.1 Click on **Create stack** and select the **With new resources (standard)** option



2.2 In the Prerequisite - Prepare template section, select **Create template in Designer** and click on the **Create template in designer** button



2.3 Under the **new template** section, click on **Template** and add the Header, Parameters, and Resource components, which are shared below:

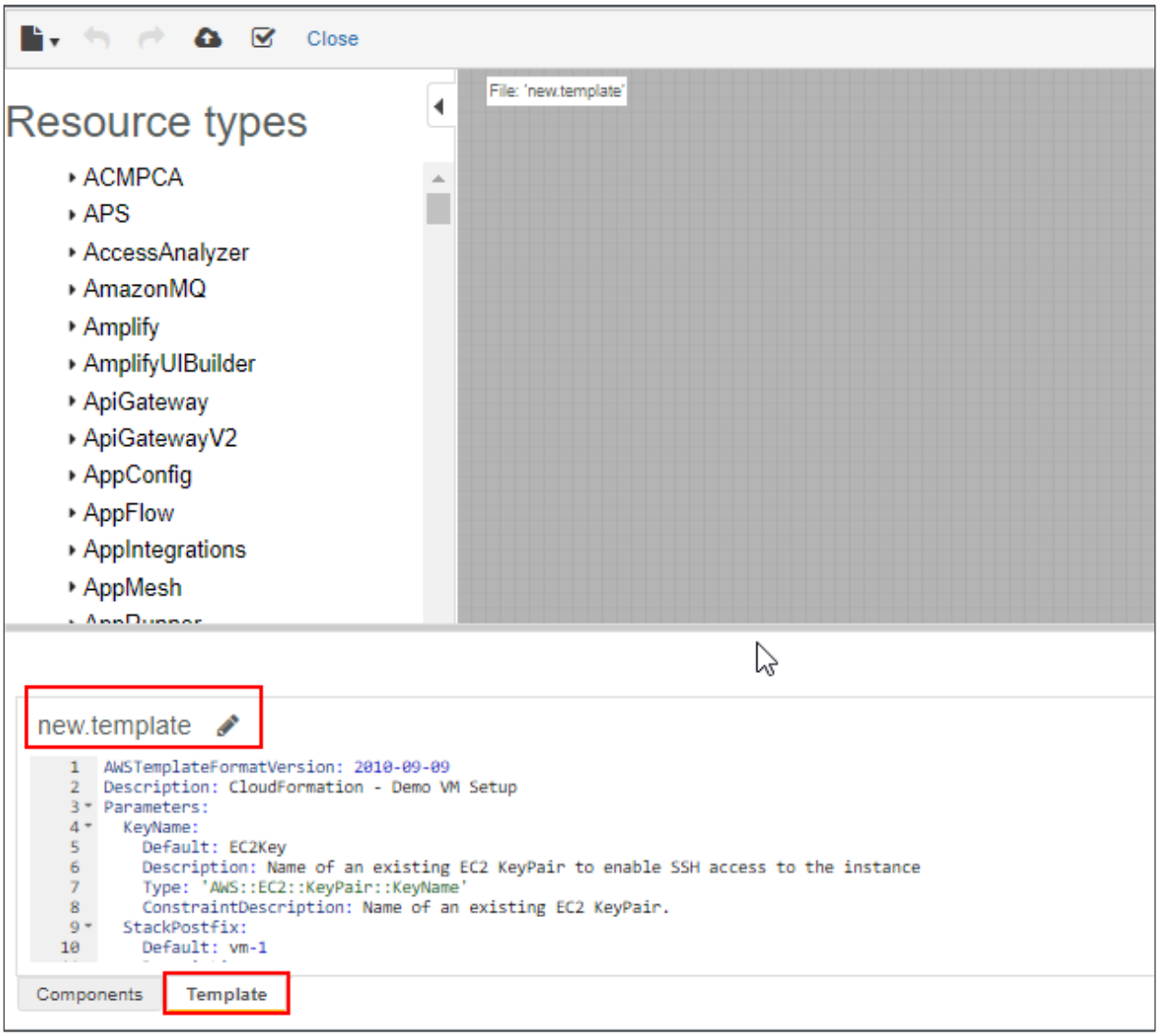
```
# CloudFormation Template for a VM

AWSTemplateFormatVersion: 2010-09-09
Description: 'CloudFormation - Demo VM Setup'

Parameters:
  KeyName:
    Default: 'EC2Key'
    Description: 'Name of an existing EC2 KeyPair to enable SSH access to the instance'
    Type: 'AWS::EC2::KeyPair::KeyName'
    ConstraintDescription: Name of an existing EC2 KeyPair.
  StackPostfix:
    Default: 'vm-1'
    Description: 'Postfix for this vm stack. This is purely for ease of use, to recognize a
particular VM more easily.'
    Type: String
    MinLength: '1'
    MaxLength: '64'
  InstanceType:
    Description: 'WebServer EC2 instance type. Change based on application needs.'
    Type: String
```

```
    Default: t2.micro
    AllowedValues:
      - t1.micro
      - t2.nano
      - t2.micro
      - t2.small
      - t2.medium
      - t2.large
    ConstraintDescription: must be a valid EC2 instance type.
  InstanceAMIId:
    Default: 'ami-0430580de6244e02e'
    Description: 'Provide AMI ID for VM'
    Type: String
  CidrBlock:
    Description: 'Enter CIDR block that the VPC should use. Example: 10.0.0.0/16'
    Type: String
    MinLength: '9'
    MaxLength: '18'
    Default: 10.0.0.0/16
    AllowedPattern: '(\d{1,3})\.(\d{1,3})\.(\d{1,3})\.(\d{1,3})/(\d{1,2})'
    ConstraintDescription: must be a valid IP CIDR range of the form x.x.x.x/x.
  LimitedSSHAccess:
    Description: 'Enter CIDR for network that may access via SSH'
    Type: String
    MinLength: '9'
    MaxLength: '18'
    Default: 0.0.0.0/0
    AllowedPattern: '(\d{1,3})\.(\d{1,3})\.(\d{1,3})\.(\d{1,3})/(\d{1,2})'
    ConstraintDescription: must be a valid IP CIDR range of the form x.x.x.x/x.

Resources:
  # Define an EC2 instance. This is our virtual machine
  VM:
    Type: 'AWS::EC2::Instance'

    Properties:
      ImageId: !Ref InstanceAMIId
      BlockDeviceMappings:
        - DeviceName: /dev/xvda
          Ebs:
            VolumeType: gp2
            VolumeSize: 200
            DeleteOnTermination: true
```

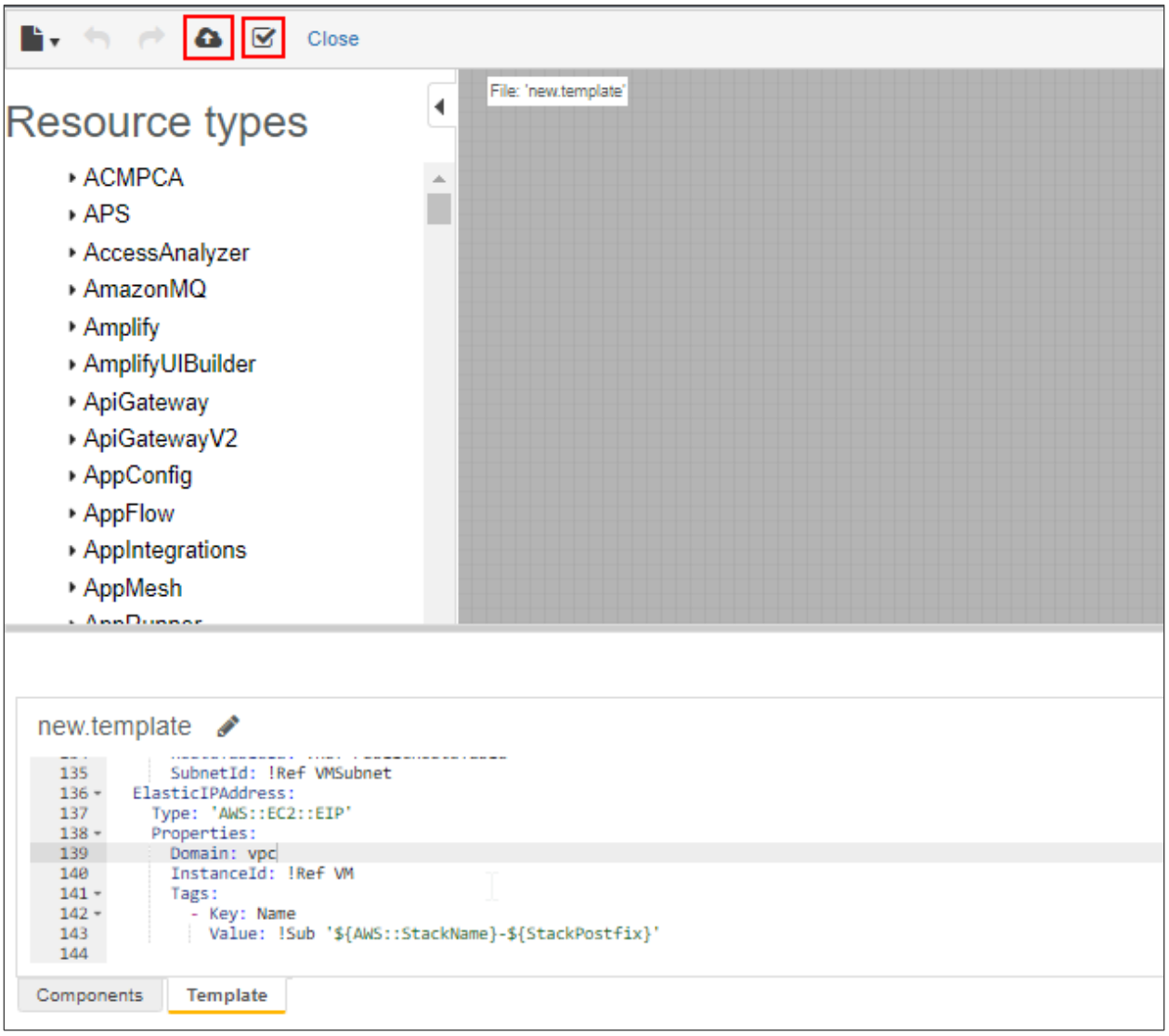
```
        Encrypted: true
    InstanceType: !Ref InstanceType
    KeyName: !Ref KeyName
    SubnetId: !Ref VMSubnet
    SecurityGroupIds:
      - !Ref FormationSecurityGroup
    Tags:
    - Key: Name
      Value: !Sub ${AWS::StackName}-${StackPostfix}

  FormationSecurityGroup:
    Type: 'AWS::EC2::SecurityGroup'
    Properties:
      GroupDescription: Enable SSH and general HTTP Access
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 80
          ToPort: 80
          CidrIp: 0.0.0.0/0
        - IpProtocol: tcp
          FromPort: 22
          ToPort: 22
          CidrIp: !Ref LimitedSSHAccess
      VpcId: !Ref VMvpc
      Tags:
      - Key: Name
        Value: !Sub ${AWS::StackName}-${StackPostfix}

  VMvpc:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: !Ref CidrBlock
      EnableDnsHostnames: true
      EnableDnsSupport: true
      InstanceTenancy: default

      Tags:
      - Key: Name
        Value: !Sub ${AWS::StackName}-${StackPostfix}

  VMInternetGateway:
    Type: AWS::EC2::InternetGateway
    Properties:
```

```
    Tags:
    - Key: Name
      Value: !Sub ${AWS::StackName}-${StackPostfix}
AttachGateway:
  Type: AWS::EC2::VPCGatewayAttachment
  Properties:
    VpcId: !Ref VMvpc
    InternetGatewayId: !Ref VMInternetGateway

VMSubnet:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VMvpc
    CidrBlock: 10.0.0.0/16
    AvailabilityZone: !Select
      - '0'
      - !GetAZs ''   # Get the first AZ in the list
    Tags:
    - Key: Name
      Value: !Sub ${AWS::StackName}-${StackPostfix}

PublicRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VMvpc
    Tags:
    - Key: Name
      Value: !Sub ${AWS::StackName}-${StackPostfix}
PublicRouteVM:
  Type: AWS::EC2::Route
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: 0.0.0.0/0
    GatewayId: !Ref VMInternetGateway

SubnetRouteTableAssociation:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    RouteTableId: !Ref PublicRouteTable
    SubnetId: !Ref VMSubnet

ElasticIPAddress:
  Type: AWS::EC2::EIP
```

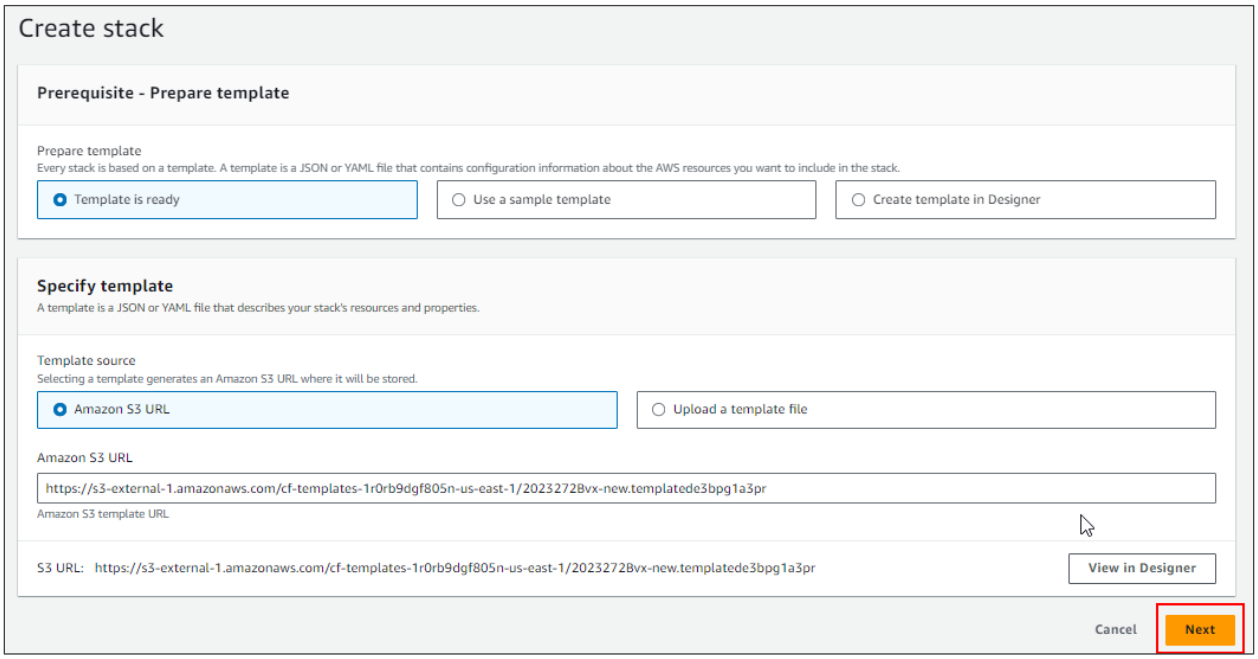
Properties:
Domain: vpc
InstanceId: !Ref VM
Tags:
- Key: Name
Value: !Sub ${AWS::StackName}-${StackPostfix}

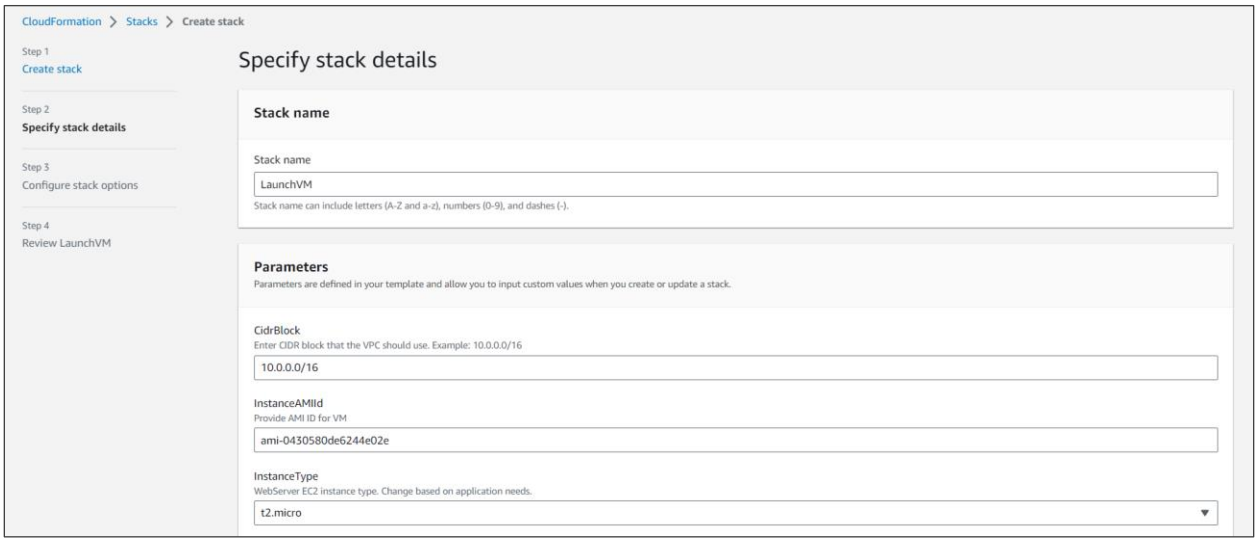


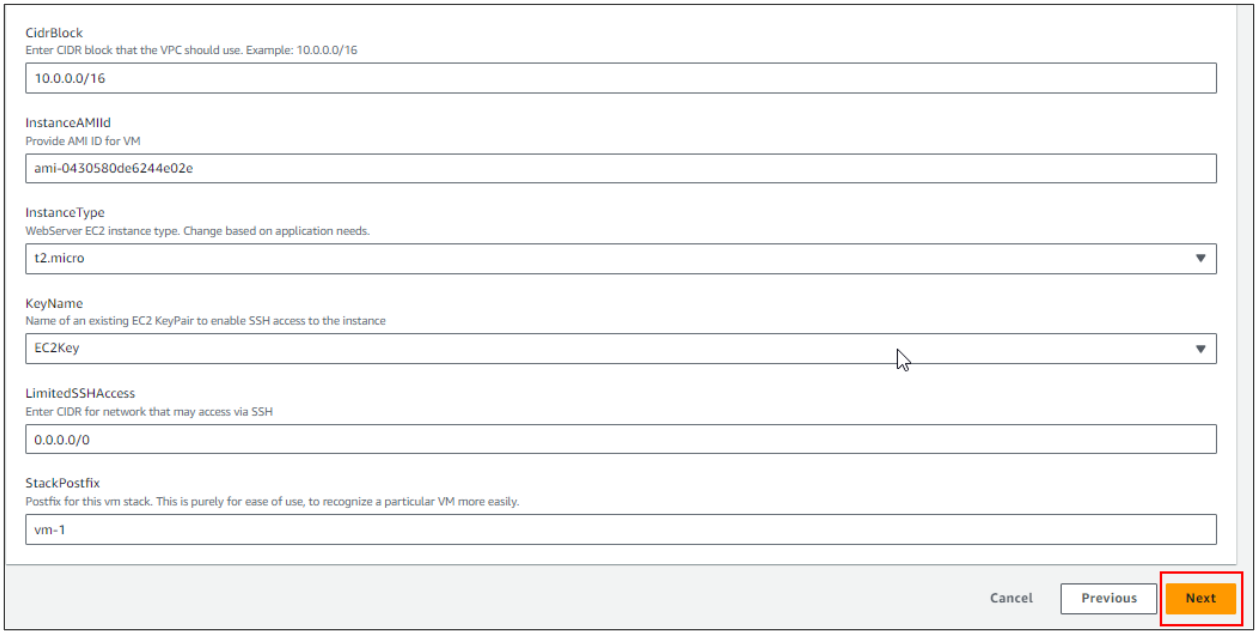
2.4 Click on the **Validate** icon to validate the template, remove any errors, and click on the **Create stack** icon



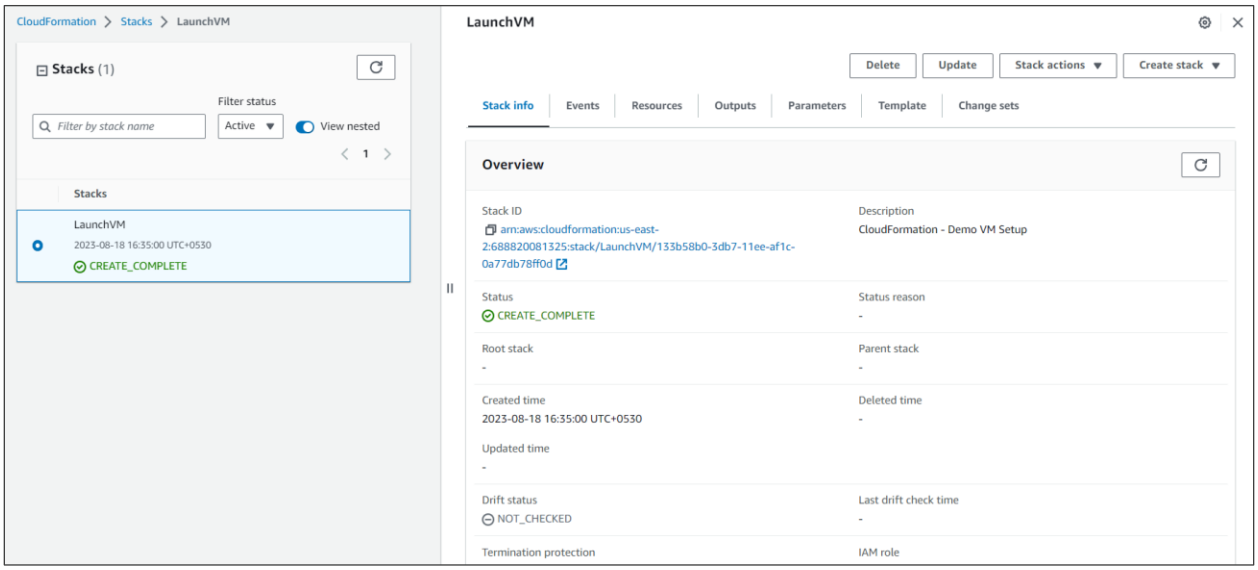
2.5 Click on the **Next** button. Add the **Name,** specify other stack details, and click **Next** per the below screenshots:



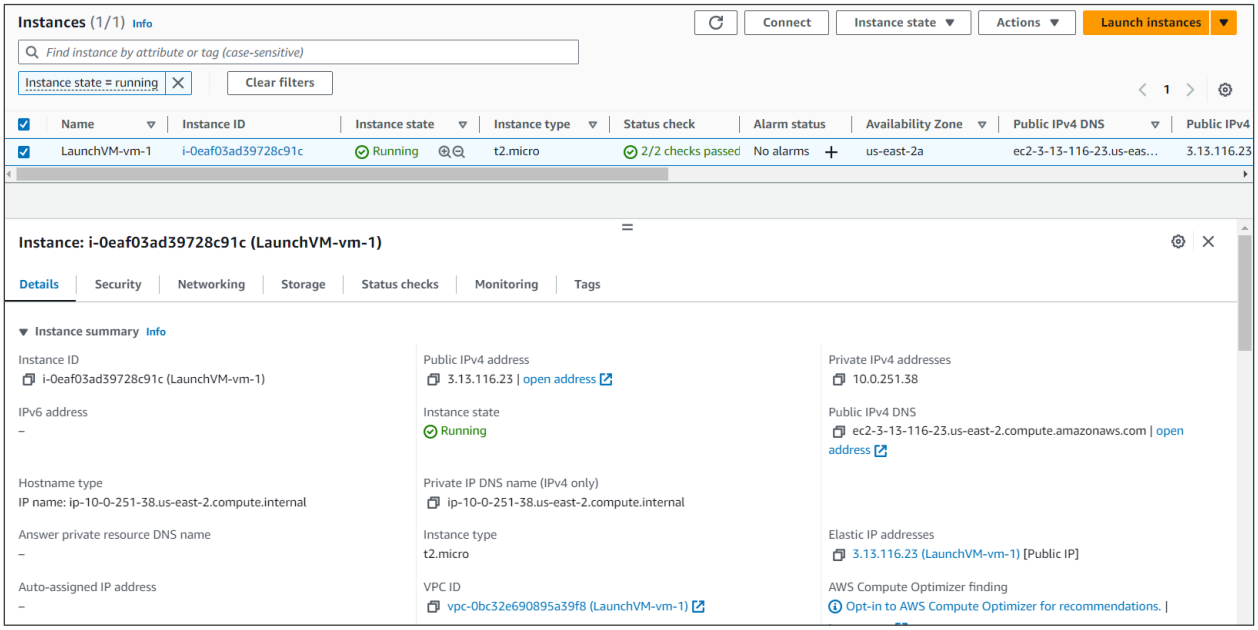




2.6 Validate whether the stack is created per the screenshot:



2.7 Navigate to EC2 services to list all the available instance



With this, we have successfully created a CloudFormation stack and launched a VM.